

# AgentDescent: Gradient Descent Where the Parameters Are Agents

A Parallel, Asynchronous, Merge-Based Framework for Self-Evolving Agent Artifacts

Birfy Chen

birfychen@gmail.com

<https://github.com/Birfy/agentdescent>

August 2026

## Abstract

Self-improving agent systems — agents that evolve their own skills, prompts, and harnesses — are today overwhelmingly *serial*: one candidate is proposed, evaluated, and accepted or rejected per iteration, bounding improvement throughput at  $O(1/T_{\text{iter}})$ . We present **AgentDescent**, a framework that transplants the deep-learning *training stack* onto agent self-evolution. The “parameters” are a library of evolvable artifacts (skills, prompts, harness modules, verifiers); the “gradients” are *diffs carrying evidence cards*; the “optimizer step” is a *merge decision* over a git-backed, version-vectored ledger.  $N$  workers propose diffs in parallel and a barrier-free asynchronous aggregator merges them, targeting  $O(N/T_{\text{iter}})$  improvement throughput. The one place the analogy must break defines the system: *gradients add, diffs do not* — so aggregation is not averaging but conflict resolution, statistical acceptance under a per-artifact Beta posterior, and transactional commit. Staleness from asynchrony is handled by a per-diff lag measure  $\eta$  with rebase-and-re-verify policies, and a layered governance model (frozen safety layer, oracle-gated harness layer, freely merging skill layer) keeps the self-referential loop from polluting itself. We demonstrate generality by porting *eighteen* published self-evolution algorithms — seven benchmark-faithful (ACE, GEPA, EvoSkill, SkillOpt, ADAS, DGM, OpenEvolve) and eleven declared microports and analogues — all running unchanged under serial, synchronous, and barrier-free asynchronous scheduling. Empirically, thread-level rollout parallelism reaches 5.8× on real API calls, dispatch scales near-linearly to 8 workers, and on a live model (GEPA/HotpotQA) removing the round barrier lifts effective concurrency from 1.85× to 3.22× over a true serial control at a pinned rollout budget. An equal-budget comparison of merge-of- $N$  against best-of- $N$  forking is reported honestly: at small budgets the two are not yet statistically separated, and we analyze why. Code, all eighteen ports, and every measured number’s reproduction command are open source.

## 1 Introduction

A rapidly growing literature shows that large-language-model agents can improve their own artifacts: their prompts [1, 6], their in-context playbooks [36], their skill libraries [28, 38], their workflows [9, 35], and even their own code [19, 21, 32, 34]. Nearly all of these systems share an execution shape inherited from their conceptual ancestor, the Gödel machine [23]: a *serial* improvement loop. One candidate modification is proposed, evaluated against a benchmark, and accepted or rejected; then the next. If one iteration takes  $T_{\text{iter}}$  wall-clock seconds, improvement throughput is bounded at one accepted change per  $T_{\text{iter}}$  — regardless of how much compute is available.

Deep learning faced the same bottleneck a decade ago and solved it with a now-standard stack: data-parallel workers computing gradients concurrently, parameter servers aggregating them [14], lock-free and

staleness-tolerant asynchronous updates [18], adaptive per-parameter step sizes [13], and gradient surgery when updates conflict [33]. The premise of this paper is that the same stack, suitably reinterpreted, applies to agent self-evolution.

**AgentDescent** realizes this premise (Fig. 1). The parameter tensor becomes a *library of evolvable artifacts*; the gradient becomes a *diff* annotated with an *evidence card* recording the rollout that motivated it; the parameter server becomes a git-backed, version-vectored *ledger*; and the optimizer step becomes an *aggregator* that merges concurrent diffs.  $N$  workers roll out tasks against a snapshot of the library and propose diffs in parallel; a barrier-free asynchronous merger aggregates them into a shared, version-controlled artifact library, targeting  $O(N/T_{\text{iter}})$  improvement throughput where the serial loop is bounded at  $O(1/T_{\text{iter}})$ .

The analogy is productive precisely because of where it breaks. *Gradients add; diffs do not*. Two gradients from different minibatches can always be summed; two textual edits to the same artifact may be complementary, redundant, or contradictory. Aggregation in AgentDescent is therefore not averaging but a discrete-space optimizer step: staleness filtering, conflict resolution, fusion of complementary edits, a statistical acceptance test under a per-artifact Beta posterior, and a transactional compare-and-swap commit (Section 4). A second disanalogy is equally load-bearing: training code is not modifiable by the gradients it computes, but a self-evolving agent system *can* rewrite its own verifier — so AgentDescent enforces a layered governance model in which safety-critical components are frozen and high-blast-radius artifacts are forced through an oracle (Section 6).

## Contributions.

1. **A systems design** that maps the deep-learning training stack — parameter server, asynchronous SGD, staleness bounds, adaptive per-parameter tests, gradient surgery, weight averaging, EMA — onto parallel agent self-evolution, with an explicit account of where the analogy holds and where it must break (Section 3).
2. **A discrete-space aggregator** implementing merge as optimization: staleness policies (Full / Guarded / Reflective) over a per-diff lag measure  $\eta$ , PCGrad-style conflict projection, model-soup-style fusion with an optional fusion tournament, Beta-posterior acceptance with annealed confidence, and dual-branch (dev/stable) promotion (Section 4).
3. **Two execution runtimes and three parallelism paradigms**: a synchronous data-parallel round loop and a barrier-free asynchronous stage-orchestration runtime with a version-lag budget (`async_ratio`) and backpressure; plus data-, tensor-, and pipeline-parallel decompositions of the artifact library (Section 5).
4. **Generality, demonstrated**: eighteen published self-evolution algorithms ported onto one engine — seven reproducing their paper’s own benchmark, eleven as declared mechanism microports and analogues — each runnable, offline-tested, and measured under all three schedulers (Section 7).
5. **An honest empirical study** (Section 8): near-linear dispatch scaling to 8 workers;  $5.8\times$  thread-level speedup on real API rollouts;  $1.85\times \rightarrow 3.22\times$  effective concurrency from removing the round barrier on a live model at a pinned budget; a staleness-policy and lag-budget characterization; and an equal-budget merge-vs-fork comparison that we report as *not yet separated*, together with the diagnosis of why separation is hard at small budgets.

## 2 Related Work

**Self-evolving agents.** Prompt evolution spans PromptBreeder’s self-referential mutation [6] and GEPA’s reflective Pareto search [1]; context/playbook evolution is exemplified by ACE [36]; skill-library growth by

**Table 1:** The central analogy. Each row of the training stack has a concrete counterpart in AgentDescent; the final row is a *disanalogy* that the design enforces rather than papers over.

| Model training                      | AgentDescent (parallel self-evolution)       |
|-------------------------------------|----------------------------------------------|
| parameter tensor $\theta$           | library of Evolvable artifacts               |
| gradient $g$                        | Diff + EvidenceCard                          |
| parameter server                    | git-backed, version-vectored Ledger          |
| optimizer step                      | Aggregator merge decision                    |
| per-parameter adaptive LR (Adam)    | per-artifact Beta-posterior acceptance test  |
| staleness / decoupled PPO           | per-diff lag $\eta$ + rebase re-verification |
| partial rollout                     | straggler detection (ResumeQueue)            |
| EMA / weight averaging              | stable/dev dual branch                       |
| training code (not self-modifiable) | L0 frozen governance layer                   |

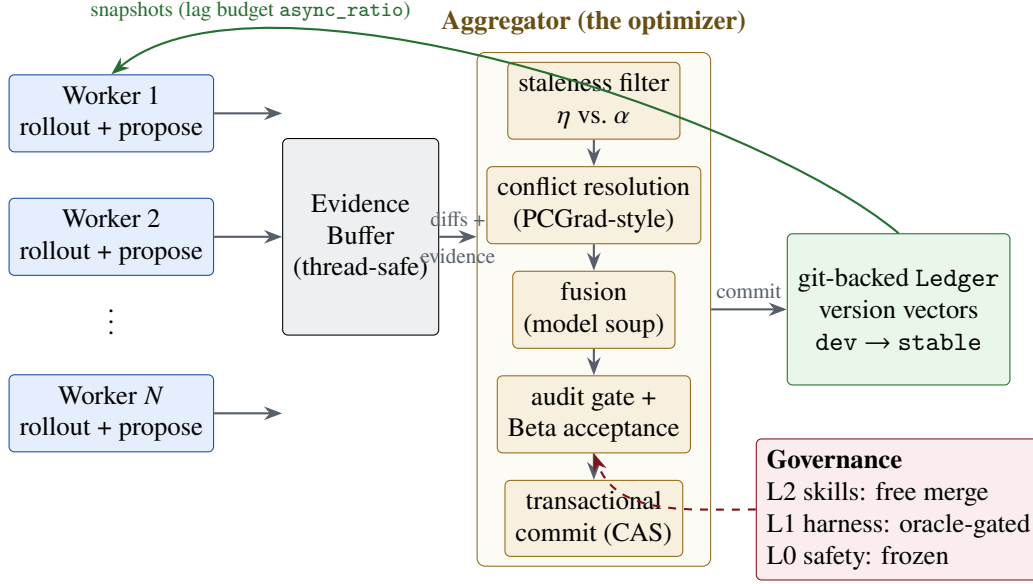
Voyager [28], SkillWeaver [38], and EvoSkill-style automated skill discovery; workflow and harness search by AFlow [35] and ADAS [9]; self-modifying code by the Darwin Gödel Machine [34], SICA [21], and Gödel Agent [32]; evolutionary program search by AlphaEvolve [19] and its open reimplementation OpenEvolve [24]; and label-free self-play by Absolute Zero [37], R-Zero [10], and Agent0 [30]. Reflection-based refinement [16, 26] supplies the inner proposal mechanism many of these share. Almost all publish a serial outer loop; AgentDescent treats that loop as one scheduling choice among three.

**Parallel and asynchronous training systems.** The parameter server [14], lock-free asynchronous SGD [18], and bounded-staleness asynchronous RL systems such as AReaL [7] and ROLL Flash [22] provide the systems vocabulary AgentDescent reuses: staleness bounds, decoupled actors and learners, and lag budgets. Adam’s per-parameter adaptivity [13], PCGrad’s conflict projection [33], and model soups [29] are the optimizer-level analogies for the per-artifact acceptance test, conflict resolution, and fusion respectively. Tensor and pipeline parallelism [11, 27] motivate the corresponding artifact-space decompositions. Concurrent work on parallelizing self-evolution (FlashEvolve-style barrier-free evolution, CoEvoSkills’ co-evolutionary verification [4]) indicates the throughput premise is a live hypothesis rather than community consensus; AgentDescent’s contribution is a narrow, defensible engineering synthesis — concurrent, staleness-bounded, conflict-resolved *diff-level merge* over a git-backed versioned ledger — with the measurement harness to test it.

## 3 The Training-Stack Analogy and System Design

### 3.1 Evolvables, diffs, and evidence cards

An *Evolvable* is any versioned artifact the system may modify: a skill document, a prompt, a playbook of bullets, a harness module, a directory of files, or program code. A *Diff* is a keyed set of operations against one or more evolvables — a *strategy* (e.g., `AppendRules`, `KeyedRules`, a per-bullet playbook, or file-path-keyed directory edits) maps a natural-language proposal into diff operations, which is what makes distinct edits fusable and conflicting edits detectable. Every diff carries an *EvidenceCard*: the task, the rollout output, the reward, and the base version the proposing worker observed. The card is the unit of statistical bookkeeping; discarded diffs settle their evidence back into the pool so information is not lost with the proposal.



**Figure 1:** AgentDescent architecture.  $N$  workers roll out tasks against ledger snapshots and emit diffs with evidence cards into a thread-safe buffer; a dedicated aggregator thread runs the five-stage merge pipeline and commits to the git-backed ledger. In the barrier-free runtime the two sides never wait for each other; a version-lag budget bounds how stale a worker’s snapshot may become. Governance (right, dashed) injects oracle audits for high-blast-radius merges and freezes the safety layer.

### 3.2 The ledger

The Ledger is the parameter server: a git-backed store with per-artifact version vectors, compare-and-swap commits, and a dual dev/stable branch pair. dev receives merges; stable is promoted only after  $K$  regression-free rounds on dev — an EMA-style confirmation in which a new commit *restarts* the clock, so the artifact most likely to be promoted is the one that stopped changing because nothing beat it. Workers read snapshots; the version vector is what makes per-diff staleness  $\eta$  computable at merge time.

### 3.3 The one required disanalogy

Because diffs do not add, every downstream design choice follows from replacing summation with a decision procedure. Averaging two contradictory edits is not meaningful; ranking and projecting them is. This is why the aggregator (next section) is best understood not as a reducer but as a discrete-space optimizer whose “step” is a merge decision, and why acceptance is statistical — a per-artifact posterior over whether the candidate improves held-out reward — rather than a learning rate.

## 4 The Aggregator as a Discrete-Space Optimizer

Evidence cards are bucketed by artifact. When a bucket triggers (batch size  $B$ , or a timeout  $T_{\max}$  so cold artifacts do not starve), the aggregator runs one optimizer step (Fig. 1, center):

1. **Staleness filter.** Each diff’s lag is  $\eta = \max(\text{head} - \text{base})$  over the artifacts it touches.  $\eta = 0$  proceeds;  $0 < \eta \leq \alpha$  is *rebased and cheaply re-verified* (does the claimed delta still hold on the new head?);  $\eta > \alpha$  is discarded, its evidence settled back into the pool. The bound  $\alpha$  adapts to artifact heat, and

contract-breaking diffs force  $\alpha = 0$ . This is the bounded-staleness discipline of asynchronous RL systems [7] applied per-diff.

2. **Conflict resolution.** Syntactic conflicts (hunk overlap) and semantic conflicts (contradictory operations) are detected; contradictions are projected out PCGrad-style [33], keeping the better of the pair as measured on a shared held-out subset.
3. **Fusion.** Complementary surviving diffs are fused — the model-soup analogy [29] — and the union proceeds to the gate. An optional *fusion tournament* ranks the union against the individual candidates on held-out data first (Section 4.1).
4. **Audit gate.** The merge decision is itself submitted to an audit scheduler: high-blast-radius or low-trust merges are forced through the oracle, which can veto outright *before* the acceptance test runs. The optimizer audits itself.
5. **Statistical acceptance.** A candidate commits only if  $P(\Delta > 0) > 1 - \delta$  under a per-artifact Beta posterior over held-out task outcomes — a posterior test rather than a point threshold, playing the role Adam’s per-parameter adaptivity plays in training [13]. The confidence requirement  $\delta$  anneals with artifact version (the learning-rate-decay analogue), and a trust region caps diff size.
6. **Commit.** Compare-and-swap on dev, one artifact per merge.
7. **Dual-branch promotion.** `dev`  $\rightarrow$  `stable` after  $K$  regression-free rounds, as above.

#### 4.1 Does merging average the improvements away?

The sharpest objection to merge-based evolution is that two individually good local edits may be jointly worse. Two properties bound the damage. First, *the acceptance gate, not the tournament, is the safeguard*: the gate scores the fused candidate on the full held-out set and refuses a measured regression, so a harmful fusion never commits. Second, the fused union is a *superset* of every individual diff, so committing it loses no proposal — it merely carries some that looked negative this round, and the next round proposes from there. The only case the tournament decides that the gate does not is a fusion that beats the base artifact but loses to one of its singles; that case is recoverable, which is why the tournament is off by default (it costs one held-out sweep per candidate, unconditionally, against a conditional and recoverable gain). It remains available (`fusion_tournament=True`) because it is the only instrument that can *measure* the objection: it reports a win rate, mean gain, the losing tail, and — critically — a contested counter for tournaments where a fused candidate was actually built and ranked. We deliberately publish no headline win rate: the quantity is a property of the artifact’s key space and of proposal overlap, not of the mechanism, and would not transfer across datasets (Section 8.6 reports it in situ for one workload).

## 5 Runtimes, Staleness, and Parallelism Paradigms

### 5.1 Two runtimes

The **synchronous data-parallel** runtime is a round barrier: all workers step, one aggregator step, next round. It is deterministic and serves as the baseline. The **asynchronous stage-orchestration** runtime removes the barrier entirely: worker threads keep producing evidence while a dedicated aggregator thread keeps merging, connected by a thread-safe evidence buffer, so the rollout/propose and aggregate/commit stages overlap instead of stalling — the partial-rollout, no-barrier discipline of asynchronous RL [7, 22] ported to self-evolution.

## 5.2 Staleness policies and the lag budget

Asynchrony’s price is staleness, and the active policy is the only thing that changes between async regimes. Given each diff’s  $(\eta, \alpha)$  the policy answers ACCEPT/REBASE/DISCARD: **Full** uses stale diffs directly ( $\eta$  ignored; maximum throughput, minimum safety); **Guarded** is version-gated bounded staleness (accept  $\eta=0$ , rebase  $\eta \leq \alpha$ , discard beyond), in the spirit of AReaL [7]; **Reflective** always rebases and re-verifies, discarding only if the delta no longer holds — recovering proposals the Guarded policy would waste. Orthogonally, `async_ratio` is a global lag budget in artifact *versions* (the ROLL-Flash-style budget [22]): a worker refreshes its snapshot only once head has drifted more than `async_ratio` versions past it, and a backpressure signal forces a global resync if evidence keeps arriving while nothing commits. Section 8.4 measures both knobs.

## 5.3 DP, TP, PP over an artifact library

Three pluggable decompositions mirror the parallelism taxonomy of large-scale training. **Data parallel** (default): same snapshot, task-sharded, diffs merged. **Tensor parallel** [27]: one *hot* artifact is split into disjoint sections, each owned by a worker, so edits are conflict-free by construction and the merge is concatenation plus a consistency review. **Pipeline parallel** [11]: artifacts form a dependency chain, and a downstream failure back-propagates blame to the earliest failing upstream stage — the same machinery used for counterfactual-replay attribution.

## 5.4 The three long tails

AgentDescent treats “the long tail” as three separate problems.  $L_{\text{traj}}$  (system): heavy-tailed rollout durations are met with an online duration estimator, longest-processing-time dispatch [8], and straggler detection; the barrier-free runtime is what actually prevents one slow rollout from stalling the rest (turn-level checkpoint-and-resume is *not* implemented — the resume queue records stragglers but nothing resumes them, since resumability would require a rollout contract the opaque run callable does not offer).  $L_{\text{task}}$  (data): Zipfian artifact triggering is met with UCB [2] over (cluster  $\times$  artifact) so starved tail artifacts receive an exploration bonus, plus a difficulty filter that down-weights all-pass/all-fail groups (the zero-advantage argument familiar from RLVR).  $L_{\text{value}}$  (signal): most diffs are marginal, so the audit scheduler spends the scarce oracle budget proportionally to blast radius  $\times$  uncertainty/trust.

# 6 Governance by Blast Radius

Artifacts sort into layers automatically by a declared `blast_radius`: **L2 (fast)** — local skills and prompts, merged under the full asynchronous pipeline; **L1 (slow)** — harnesses and verifiers, with serialized in-flight changes, staged rollout, and every merge forced through the oracle; **L0 (frozen)** — the oracle itself, the audit budget, merge permissions, and safety constraints, read-only to the loop. The frozen layer is the enforced form of the training-stack disanalogy in Table 1: without it, the self-referential loop eventually pollutes itself (the canonical failure being a verifier that learns to pass itself). For directory-shaped artifacts the same layering yields three entry points — skill directories (L2), agent definitions (L1), and executed agent *code*, which runs behind a frozen test suite that the candidate cannot rewrite (pristine test files are overlaid after materialization, so weakening the tests at run time also fails).

## 7 Eighteen Algorithm Ports

A framework claiming generality should be measured by what it can express. Eighteen published self-evolution algorithms run on the engine — none bypasses it — via two routes: the seven *benchmark-faithful* ports are each a custom `strategy` and/or `aggregator_factory` plugged into `evolve()` (e.g., GEPA’s per-instance Pareto optimizer [1], ADAS’s keep-all archive [9], DGM’s archive-plus-selection [34], OpenEvolve’s MAP-Elites islands [17, 24]); the eleven *microports and analogues* are declarative policy definitions over one shared runner, so the runtime matrix can vary only the scheduler. Fidelity is to the *released code*, not just the paper (e.g., EvoSkill’s frontier is a top- $K$  aggregate, not a per-instance Pareto front, and the port follows the code and says so); every port carries a fidelity class — *benchmark-faithful*, *mechanism microport*, *self-edit analogue*, *environment analogue*, or *inference analogue* — and analogues must not be cited as paper-benchmark reproductions. Every mechanism family in the field’s current taxonomy (evolutionary search, reflection, skill libraries, self-play, self-modifying code) is covered by at least one faithful port or one declared analogue. All eighteen run serial, synchronous-parallel, and barrier-free asynchronous without modification, because their optimizers keep shared state thread-safe.

## 8 Experiments

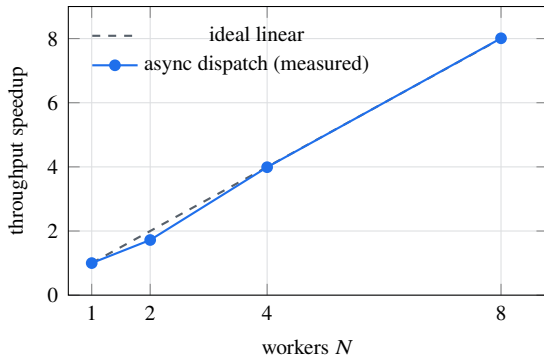
All numbers below are produced by scripts shipped in the repository and named beside each result; absolute times depend on the host and endpoint, so we report ratios and provide the exact reproduction command for every row. Unless stated otherwise, live-model runs use `deepseek-v4-flash` or GLM-5.2 through an OpenAI-compatible endpoint.

### 8.1 Is thread-level rollout parallelism real?

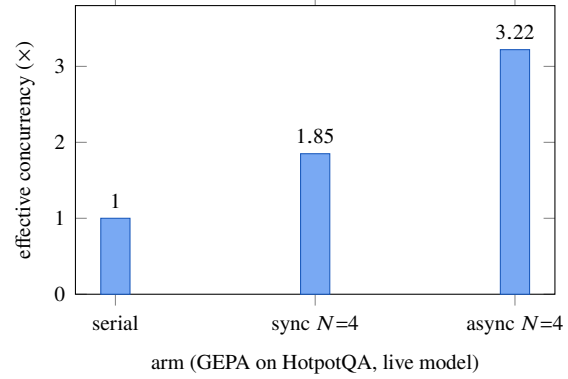
Agent rollouts are I/O-bound (the worker waits on a model), so Python threads should overlap them despite the GIL — but this is measured, not assumed. Eight threads over one pool, two workloads: a real glm-5.2 API round trip falls from 48.6 s sequential to 8.3 s (**5.8×**; 5.8–6.5× across three runs, sub-linear because a reasoning model’s heavy-tailed latency means eight threads finish when the slowest finishes), while pure-Python CPU arithmetic gains nothing (1.1×). The corollary matters more than the headline: the speedup tracks how much of a rollout is *waiting*, so CPU-heavy agents belong behind a process pool while the framework’s workers stay on the I/O.

### 8.2 Dispatch scales near-linearly; the barrier does not

Two complementary measurements separate what parallelism buys from what the barrier costs. First, *dispatch*: running the barrier-free runtime for a fixed wall-clock window and counting rollouts gives near-linear scaling through 8 workers (Fig. 2); the rollout stage holds no global lock, so this is the  $O(N/T_{\text{iter}})$  regime the design targets. Second, *end-to-end*: a whole `evolve()` run at default settings reaches only 1.8× with 8 workers under uniform rollout latency, and 1.7× under a heavy-tailed (reasoning-model-like) latency — because the round barrier waits for the slowest worker and, past that, the ceiling is whatever in a round is *not* a rollout, which at default settings is the acceptance gate. The two gaps have two independent knobs: the gate has its own concurrency pool (`eval_concurrency` 1  $\rightarrow$  8 takes the gate from 3.6 s to 1.2 s, saturating at the held-out size), and the barrier can be removed — on a heavy-tailed dispatch microbenchmark the asynchronous pipeline runs at  $\sim 100\%$  utilization against the barrier’s 36–40%, a 2.65× wall-clock speedup (robust range 2.57–2.93×), tracking  $\mathbb{E}[\max \text{ of } N] / \mathbb{E}[\text{latency}]$ .



**Figure 2:** Rollout-dispatch throughput of the barrier-free runtime vs. worker count (fixed wall-clock window; `examples/efficiency.py`, Experiment 1). Speedup is near-linear to 8 workers — the rollout stage holds no global lock. Across repeated runs the 8-worker figure landed between 7.8× and 9.2×; the honest reading is “linear to within measurement noise at this scale.”



**Figure 3:** Effective concurrency (model-seconds ÷ wall-clock) on a faithful GEPA port, HotpotQA, `deepseek-v4-flash`, 16 rollouts pinned on every arm. The serial control is the published loop exactly (one worker, serial gate: overlap 1.00×). The round barrier is the whole difference between the parallel arms: the same four workers reach 1.85× with it and 3.22× without it. Wall-clock: 609 s / 239 s / 140 s.

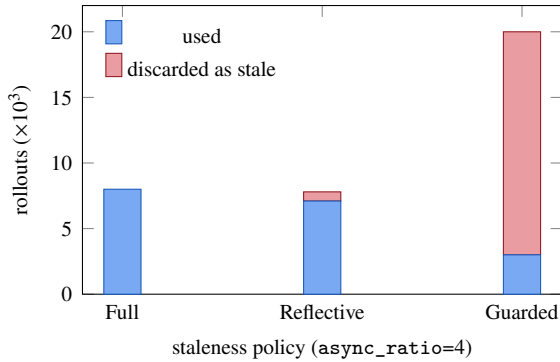
### 8.3 On a live model, against a true serial control

Stub latencies can flatter a scheduler, so the same question is put to a faithful port of a published serial algorithm on its own benchmark: GEPA on HotpotQA [1, 31], 16 rollouts pinned on every arm, and a control that is the upstream loop *exactly* — one worker *and* a serial gate (a control that still evaluates concurrently is already partly parallel; this one overlaps at precisely 1.00×). Figure 3 shows the result: the synchronous arm reaches 1.85× effective concurrency — consistent with the heavy-tail stub prediction of 1.7–1.8× — and removing the barrier takes the same four workers to 3.22×. Wall-clock falls faster than concurrency rises (609 s → 239 s → 140 s) because model-seconds per rollout also fall (37.9 → 27.7 → 23.8): reflective merging collapses each round’s diffs into one gate sweep, and the async arm discards stale proposals before they reach the gate. That saving is real but is not parallelism, so it is reported beside the concurrency figure, not inside it. Two caveats: the async arm overran the pinned budget (19 rollouts vs. 16) because a barrier-free path has no round boundary at which to stop, and at `async_ratio=3` this workload discarded every stale proposal the filter saw — buying wall-clock and spending proposals.

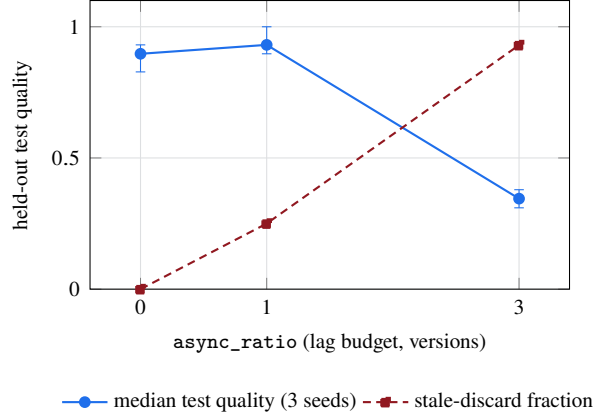
A second live-model experiment isolates merging as a *cost* lever at width 8 (GEPA’s own Pareto optimizer untouched, empty seed instruction, 16 rollouts pinned, one seed, wall-clock net of endpoint failures): serial 1424 s / 97 calls, sync  $N=8$  1022 s / 70 calls, async  $N=8$  742 s / 95 calls. The merge lever works exactly where admissions are the cost — sync fuses six single-candidate admissions into two merged ones (−28% calls) — while async gives most of that saving back (its merger sweeps on its own cadence, so batches shrink) but wins wall-clock (1.92×). Raw `curl` probes put the endpoint’s own ceiling at 7×–10×, so the measured ladder (1.31×/1.80×/2.89× effective concurrency) is the engine’s structure, not the provider’s.

### 8.4 Staleness policies and the lag budget

Figure 4 compares the three policies at a deliberately large lag budget: all converge, and the cost structure is the message — Guarded pays for safety in discarded work, Full pays nothing on a domain whose diffs happen to compose, and Reflective buys most of Guarded’s safety at a fraction of its waste by rebasing instead of discarding. Figure 5 sweeps the lag budget itself on the deterministic reference domain and exposes a genuine



**Figure 4:** The three staleness policies at a large lag budget (examples/run\_async). All three converge to reward 1.000 on the reference domain; what differs is the cost. Guarded discards ~85% of its rollouts as stale (~5.1 s wall vs. ~3.2 s); Reflective recovers most would-be-stale proposals by rebasing and re-verifying. Ratios are the result — absolute counts scale with the host.



**Figure 5:** The lag budget is a budget in artifact *versions*, so its right value depends on rollout cost. On the microsecond-rollout reference domain (async  $N=4$ , 3 seeds, commit-resync pinned off), `async_ratio=3` discards 93% of evidence and never reaches the quality bar; 1 comes within a point of synchronous; 0 is the steadiest row. The default (3) is tuned for seconds-long model rollouts, and a run that discards more than half its evidence now warns at runtime.

footgun that the framework now warns about at runtime: `async_ratio` is denominated in artifact versions, so on a domain where a rollout costs microseconds a worker drifts past any budget almost instantly — at lag 3 the run discards 93% of its evidence and never reaches the quality bar, while at lag 1 it comes within a point of the synchronous path. The default stays tuned for the workload the framework is for (seconds-long model rollouts), not the one that is cheap to measure. The same sweep also shows that *parallelism buys time, not rollouts*: from 1 to 8 synchronous workers, time-to-quality falls  $1.26 \rightarrow 0.52 \rightarrow 0.25$  (normalized) while cost-to-quality *rises* slightly ( $22 \rightarrow 24$  rollouts), and reach-rate improves from 2/3 to 3/3 seeds. These coordination-cost numbers involve no model call and are evidence the harness works, not an answer to the parallelism question — that answer is Section 8.3.

## 8.5 Do the ports still learn? The runtime-matrix quality results

Speed is worthless if the ports stop learning under the parallel schedulers, so every port is also measured for quality. Table 2 and Fig. 6 report the eleven declarative ports under the *most aggressive* configuration — barrier-free, 8 workers, Full staleness — at three seeds each: all eleven improve their held-out test metric, with gains bounded by each benchmark’s headroom exactly as they should be (the GSM8K rows converge on 0.94–0.97 and separate on *cost*: AFlow spends twice Self-Refine’s calls for 0.026 more). Acceptance is sparse by design — 3–10 committed proposals out of 240 — because the gate admits only strict held-out improvements. Table 3 reports the seven benchmark-faithful ports on their own benchmarks. An earlier fixed-fingerprint run of the full runtime matrix (11 methods  $\times$  3 schedulers, paired over  $n=33$ ) measured a median  $1.36\times$  end-to-end and  $1.89\times$  engine-window speedup of synchronous-parallel over serial, with 11/11 methods showing a median win.

Beyond the ports, the highest-level one-call path (`evolve_skill`) on 40 real HotpotQA rows lifts held-out exact match from 0.167 to 0.583 in four rounds (338 calls, one committed proposal, three rejected below threshold), learning the instruction “*Respond with only the requested answer, omitting any extra explanation or restatement*”; a bring-your-own-agent run on word problems scored in undocumented integer-cents

**Table 2:** The eleven microports and analogues on one runner and one budget: 3 seeds each, barrier-free async pipeline, 80 rollouts, 8 workers, Full staleness, deepseek-v4-flash. “moved” counts seeds whose held-out test score rose; “accepted” is committed proposals out of 240 across the three seeds. Rows are comparable only within a domain: a gain is bounded by the headroom its baseline leaves (GSM-Hard’s noise floor is  $\pm 0.02$  vs. GSM8K’s  $\pm 0.09$ ).

| Method             | Domain          | test, before $\rightarrow$ after | gain   | moved | accepted | calls/seed |
|--------------------|-----------------|----------------------------------|--------|-------|----------|------------|
| PromptBreeder [6]  | GSM8K           | 0.474 $\rightarrow$ <b>0.969</b> | +0.495 | 3/3   | 8/240    | 1444       |
| AFlow [35]         | GSM8K           | 0.510 $\rightarrow$ <b>0.969</b> | +0.458 | 3/3   | 10/240   | 2069       |
| Self-Refine [16]   | GSM8K           | 0.552 $\rightarrow$ <b>0.943</b> | +0.391 | 3/3   | 5/240    | 1050       |
| Reflexion [26]     | GSM-Hard        | 0.583 $\rightarrow$ <b>0.599</b> | +0.016 | 2/3   | 3/240    | 465        |
| SICA [21]          | GSM-Hard        | 0.641 $\rightarrow$ <b>0.667</b> | +0.026 | 2/3   | 5/240    | 899        |
| Gödel Agent [32]   | GSM-Hard        | 0.625 $\rightarrow$ <b>0.687</b> | +0.062 | 2/3   | 9/240    | 985        |
| Voyager [28]       | crafting world  | 0.000 $\rightarrow$ <b>0.667</b> | +0.667 | 2/3   | 3/240    | 1234       |
| SkillWeaver [38]   | settings site   | 0.000 $\rightarrow$ <b>0.771</b> | +0.771 | 3/3   | 8/240    | 1170       |
| Absolute Zero [37] | self-play carts | 0.042 $\rightarrow$ <b>0.354</b> | +0.313 | 3/3   | 9/240    | 624        |
| R-Zero [10]        | self-play carts | 0.062 $\rightarrow$ <b>0.292</b> | +0.230 | 3/3   | 6/240    | 937        |
| Agent0 [30]        | self-play carts | 0.042 $\rightarrow$ <b>0.542</b> | +0.500 | 3/3   | 6/240    | 1626       |

convention goes 0.250  $\rightarrow$  1.000 in one round by writing the convention down — generalizing rather than memorizing.

## 8.6 Merge-of- $N$ against best-of- $N$ fork at equal budget

Every number above is a throughput number, and throughput cannot distinguish *merging* from *sampling-and-selecting*:  $N$  independent forks saturate  $N$  workers just as well. The distinguishing quantity is held-out quality at an *equal rollout budget*, merge-of- $N$  against best-of- $N$  fork, with the fork reported both as the dev-selected arm (what fork-and-select actually delivers) and the test-oracle arm (an unshippable upper bound). On HotpotQA through GLM-5.2 (80 rows split 40/20/20, 9 rollouts pinned,  $N=3$ , 3 seeds, reflective merge so contradicting proposals survive to be fused), the result is a **null**: merge-of-3 has the highest median test EM (0.800 vs. 0.700 for both serial and fork) *and* the widest spread (0.600–0.850), and the intervals overlap — the framework’s own separation test refuses to call it a win, which is what it is for. Two secondary observations are still informative: the call confound runs in merge’s favor (73 calls vs. serial’s 93 and fork’s 132, because a merge arm shares what its workers learned while forks each start from nothing), and fork pays a real price for choosing on dev (oracle median 0.750 vs. selected 0.700). The binding caveat is mechanistic: at this budget the fused union was built *twice in the whole experiment* — most merges saw a single surviving diff — so this is not evidence that merging does not help; it is an experiment in which merging barely fired. A related subtlety generalizes: an artifact whose strategy keeps all state in one key (GEPA’s single instruction slot) makes every pair of proposals contradict by construction, reducing merge-of- $N$  to per-round best-of- $N$  selection unless a reflective model-written merge is installed; the contested counter exists so such runs cannot be misread as merge wins. Larger budgets on multi-key artifacts are the open measurement.

## 9 Scope, Limitations, and Honest Reporting

AgentDescent is a research reference implementation, not a production system, and several limits are stated rather than hidden. (i) The throughput premise is a *testable engineering hypothesis*, not community consensus; the equal-budget quality question (Section 8.6) is currently a null at small budgets. (ii) Turn-level checkpoint-and-resume of stragglers is not implemented, and neither is a stratified tail-canary held-out split; the paper’s tables mark both. (iii) Most quality rows carry three seeds and the seven faithful ports carry one — evidence the mechanism runs and moves the metric it should, not paper-scale claims. (iv) Difficulty knobs

**Table 3:** The seven benchmark-faithful ports, each on its own benchmark. Single runs (one seed) that establish each port works end to end — not a cross-method comparison, and not paper-scale results.

| Method          | Domain                                            | Budget                       | Measured                                                                                                                           |
|-----------------|---------------------------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| ACE [36]        | FiNER-139 [15]                                    | async $N=4$ , 120 roll-outs  | val 0.719 $\rightarrow$ <b>0.766</b> , test 0.786; 10 bullets curated                                                              |
| GEPA [1]        | HotpotQA [31]                                     | 16 rollouts, all arms pinned | $1.85 \times / 3.22 \times$ concurrency (sync/async) vs. true serial; test EM 0.600 $\rightarrow$ 0.850 on 20 items                |
| EvoSkill        | FinQA [3]                                         | async $N=4$ , 120 roll-outs  | val 0.527 $\rightarrow$ <b>0.707</b> , test 0.633; frontier filled 5/5                                                             |
| SkillOpt        | SearchQA [5] (hard subset)                        | async $N=4$ , 60 roll-outs   | val 0.053 $\rightarrow$ <b>0.211</b> , test 0.316; 3 of 43 edits accepted                                                          |
| ADAS [9]        | MGSM [25], GPQA [20]                              | —                            | no lift number: MGSM saturated; GPQA at 49 s / 5,116 tokens per call                                                               |
| DGM [34]        | vendored bugs, real pytest (SWE-bench-style [12]) | async $N=2$ , 16 roll-outs   | seed agent 0.844 $\rightarrow$ best archived child <b>0.906</b> held out; its <code>solve.py</code> grew 18 $\rightarrow$ 79 lines |
| OpenEvolve [24] | function minimization                             | async $N=4$ , 24 roll-outs   | combined score 0.9638 $\rightarrow$ <b>1.4995</b> vs. a 1.5 ceiling, held-out seeds                                                |

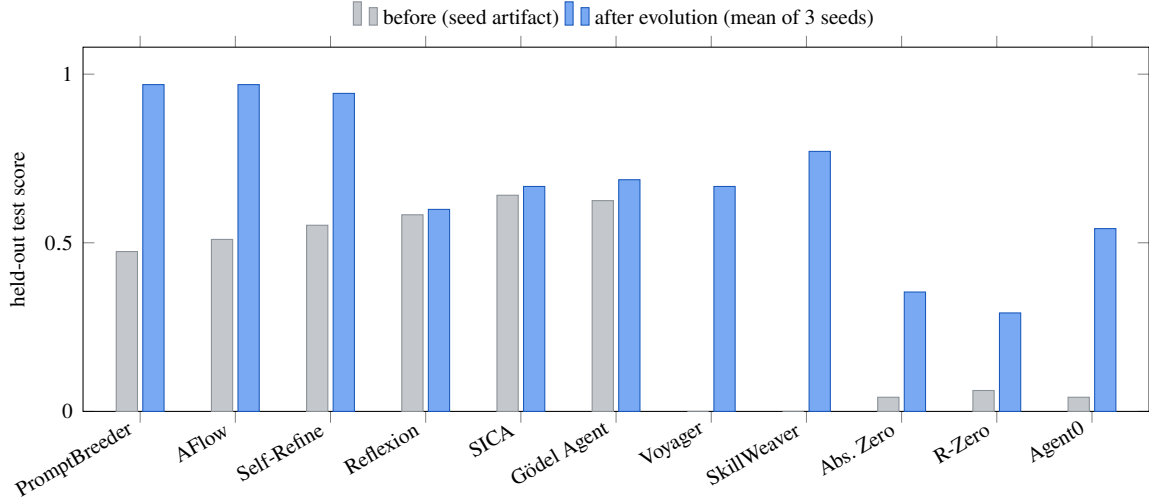
calibrated on one model do not transfer: re-measured under a model swap at fixed configuration, the two ports whose difficulty was a knob (ACE’s top- $k$ , SkillOpt’s hard-subset selector) lost their lift entirely — the mechanism ran correctly and there was nothing left to learn — while the ports whose difficulty is a property of the benchmark reproduced. (v) Several earlier headline numbers in the project’s own history were found to be irreproducible (no generating script) or measured with flattering definitions (setup time in the denominator, double-counted staleness denominators, one-run-per-arm comparisons), and were re-measured and corrected in place; every number in this paper is generated by a named script. We consider this correction trail part of the contribution: a framework for self-improving systems should hold its own evaluation to at least the standard its acceptance gate holds diffs to.

## 10 Conclusion

AgentDescent shows that the deep-learning training stack — parameter server, asynchronous data parallelism, bounded staleness, per-parameter adaptive acceptance, gradient surgery, weight averaging, and a frozen training substrate — has a coherent and implementable counterpart in agent self-evolution, once the one necessary disanalogy (diffs do not add) is taken seriously and merge is built as a discrete-space optimizer. Eighteen published algorithms run unchanged on the engine under serial, synchronous, and barrier-free scheduling; parallelism delivers the wall-clock it promises on live models ( $1.85 \times$  with a barrier,  $3.22 \times$  without, against a true serial control); and the framework ships the instruments — fusion tournaments, contested counters, equal-budget baseline arms, separation tests — to answer the question that throughput cannot: whether merging is a new mechanism or an engineering convenience. On present evidence it is an engineering win with the quality question open, and the harness to close that question is now a `pip install` away.

## Acknowledgments

The algorithm ports were contributed by chendanyang (the seven benchmark-faithful ports) and cyanneko (OpenEvolve and the eleven microports and analogues).



**Figure 6:** Held-out test score before and after evolution for the eleven microports and analogues (Table 2): 3 seeds each under the barrier-free async pipeline, 80 rollouts, 8 workers. Between 3 and 10 of 240 proposals commit per row — the acceptance gate admits only strict held-out improvements, so low acceptance is the shape of the mechanism, not a low yield.

## Reproducibility

All code is at <https://github.com/Birfy/agentdescent> (MIT license; `pip install agentdescent`; the core engine has zero required dependencies). Every measured number in this paper names its generating script: `examples/efficiency.py` (Sections 8.1 and 8.2), `bench/matrix_run.py` and `examples/gepa/gepa_prompt_evolution.py` (Section 8.3), `examples/run_async.py` and `bench/run.py` (Section 8.4), `bench/candidate_methods.py` (Section 8.5), and `bench/baselines_run.py` (Section 8.6). Every port supports `--dry-run` (no API key, no model call) and carries an offline test suite; datasets are fetched from canonical sources by a dependency-free data layer. Absolute timings depend on host and endpoint; the repository documentation states, for each table, whether ratios or absolute values are the reproducible quantity.

## References

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [2] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2):235–256, 2002.
- [3] Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, et al. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of EMNLP*, 2021.
- [4] CoEvoSkills authors. Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026.

- [5] Matthew Dunn, Levent Sagun, Mike Higgins, V Ugur Guney, Volkan Cirik, and Kyunghyun Cho. SearchQA: A new Q&A dataset augmented with context from a search engine. *arXiv preprint arXiv:1704.05179*, 2017.
- [6] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- [7] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- [8] Ronald L Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.
- [9] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [10] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Fang, Jiaxin Huang, Tony Sanders, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data. *arXiv preprint arXiv:2508.05004*, 2025.
- [11] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in Neural Information Processing Systems*, 32, 2019.
- [12] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2023.
- [13] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [14] Mu Li, David G Andersen, Jun Woo Park, Alexander J Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J Shekita, and Bor-Yiing Su. Scaling distributed machine learning with the parameter server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 583–598, 2014.
- [15] Lefteris Loukas, Manos Fergadiotis, Ilias Chalkidis, Eirini Spyropoulou, Prodromos Malakasiotis, Ion Androutsopoulos, and Georgios Paliouras. FiNER: Financial numeric entity recognition for XBRL tagging. In *Proceedings of ACL*, 2022.
- [16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.
- [17] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [18] Feng Niu, Benjamin Recht, Christopher Ré, and Stephen J Wright. HOGWILD!: A lock-free approach to parallelizing stochastic gradient descent. *Advances in Neural Information Processing Systems*, 24, 2011.

- [19] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [20] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- [21] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- [22] ROLL Team. ROLL Flash: Accelerating RLVR and agentic training with asynchrony. *arXiv preprint*, 2025.
- [23] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs/0309048*, 2003.
- [24] Asankhaya Sharma. OpenEvolve: an open-source evolutionary coding agent. <https://github.com/codelion/openevolve>, 2025.
- [25] Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, et al. Language models are multilingual chain-of-thought reasoners. *arXiv preprint arXiv:2210.03057*, 2022.
- [26] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.
- [27] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [29] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pages 23965–23998, 2022.
- [30] Peng Xue, Cheng Wu, Zhen Lin, et al. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint*, 2025.
- [31] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.
- [32] Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.
- [33] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 33:5824–5836, 2020.

- [34] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.
- [35] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- [36] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.
- [37] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, et al. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.
- [38] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.